

OrderCraft – Manufacturing Order Management System (OMS)

Software Requirements Specification (SRS) & Project Documentation |

B.Tech Computer Science & Engineering

1. Project Overview

1.1 Introduction

OrderCraft is a web-based Manufacturing Order Management System that manages the complete manufacturing order lifecycle in a single platform: from customer order capture, BOM-based material requirement calculation, inventory check, shortage detection and procurement, to production, quality check, invoicing, payment and order closure. It is built with an Angular frontend, a layered Spring Boot backend and a MySQL database, and secures access through JWT authentication and role-based authorization for six organizational roles.

1.2 Problem Statement

Manufacturing organizations often handle orders, BOMs, stock, purchasing, production and billing in separate tools or manual records. This causes manual material-requirement calculation, late shortage detection, weak traceability between orders, purchase orders and invoices, and no unified audit history. OrderCraft integrates these activities into one controlled workflow.

1.3 Project Objectives

ID	Objective	ID	Objective
O-1	Manage customers, products and BOMs centrally	O-5	Support purchase orders, suppliers and goods receipt
O-2	Capture and validate customer orders	O-6	Track production and quality checks
O-3	Calculate material requirements from the BOM	O-7	Generate invoices and record payments
O-4	Check inventory and detect shortages	O-8	Enforce role-based access with audit logging

1.4 Project Scope

In Scope	Out of Scope (see FUTURE SCOPE, Section 10)
All 18 core modules: authentication, customer, product, BOM, order, material requirement, inventory, stock movement, supplier, purchase order, goods receipt, production, quality check, invoice, payment, dashboard, reports, audit	AI-based demand forecasting, barcode/QR integration, mobile application, cloud deployment, advanced analytics, supplier performance analytics, automated notifications, advanced production planning
Web application for desktop/laptop browsers with six role-based user types	Any hardware, IoT or embedded integration

1.5 Target Users

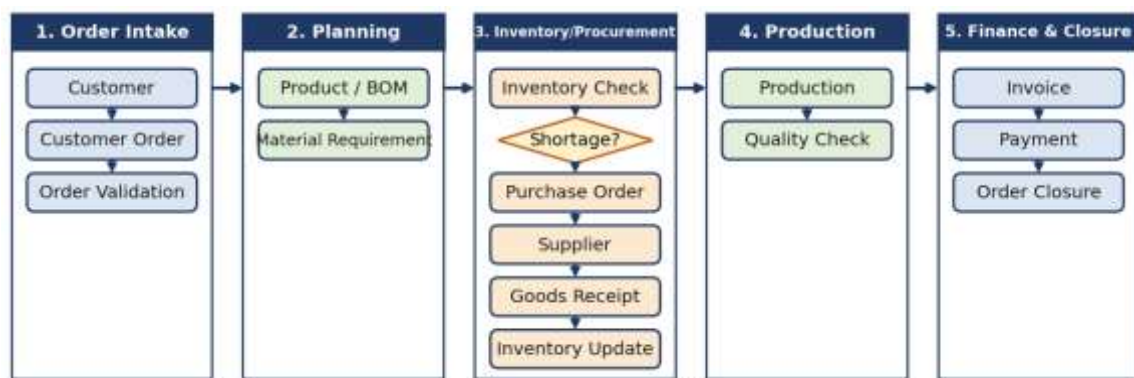
Role	Primary Responsibility	Role	Primary Responsibility
ADMIN	User and audit administration	PROCUREMENT_MANAGER	Suppliers and purchase orders
SALES_MANAGER	Customers and customer orders	WAREHOUSE_MANAGER	Inventory, stock, goods receipt

Role	Primary Responsibility	Role	Primary Responsibility
PRODUCTION_MANAGER	Production, status, quality check	FINANCE_MANAGER	Invoices and payments

1.6 Technology Stack

Layer	Technologies
Frontend	Angular, TypeScript, HTML5, CSS3, Angular Router, Reactive Forms, HttpClient
Backend	Java, Spring Boot, Spring Web, Spring Data JPA, Hibernate, Spring Security, JWT, Bean Validation, REST APIs, Maven
Database	MySQL
Security	JWT authentication, BCrypt password hashing, role-based access control, input validation, audit logging

1.7 High-Level System Workflow



2.2 Design and Implementation Constraints

ID	Constraint	Description
C-1	Layered architecture	Controller → Service → Repository separation of concerns
C-2	REST-based communication	Frontend and backend communicate only through REST APIs (JSON)
C-3	Angular frontend	User interface is implemented in Angular with TypeScript
C-4	Spring Boot backend	Business logic and APIs are implemented in Spring Boot
C-5	MySQL database	All persistent data is stored in MySQL
C-6	JWT authentication	Access to protected resources requires a valid JWT
C-7	Role-based authorization	Features and APIs are restricted by user role
C-8	Input validation	Bean Validation on the backend and Reactive Forms on the frontend
C-9	Maintainable modular code	Code is organized by module with consistent layering

2.3 Assumptions and Dependencies

Assumptions	Dependencies
A-1 Users have a modern web browser and network access	D-1 Spring Boot and the Spring ecosystem (Web, Data JPA, Security)
A-2 Users are assigned one of the six defined roles	D-2 Angular framework and Node.js toolchain
A-3 BOM data exists for a product before material requirements are calculated	D-3 MySQL server for data storage
A-4 Master data (customers, products, suppliers) is entered before transactions	D-4 Hibernate / JPA for persistence
A-5 The system is used as a web application on desktop/laptop devices	D-5 Maven for backend dependency management and build
A-6 Users keep their credentials confidential	D-6 JWT library for token generation and validation

3. System Features and Functional Requirements

Requirements use the identifier format FR-<module>.<number>. Priority: **High** = essential to the core order lifecycle or security; **Medium** = supports operation and monitoring.

3.1 Requirement Summary

No.	Module	Requirement IDs	Requirements
1	Authentication & Access Governance	FR-1.1 – FR-1.5	5
2	Customer & Order Management	FR-2.1 – FR-2.4	4
3	Product & BOM	FR-3.1 – FR-3.4	4
4	Inventory & Procurement	FR-4.1 – FR-4.7	7
5	Production	FR-5.1 – FR-5.4	4
6	Finance	FR-6.1 – FR-6.4	4
7	Reporting & Administration	FR-7.1 – FR-7.4	4
	Total		32

MODULE 1 — Authentication & Access Governance

Requirement ID	Requirement	Description	Priority
FR-1.1	Secure Login	The system shall allow registered users to log in with their credentials.	High
FR-1.2	Credential Validation	The system shall validate credentials against stored BCrypt-hashed passwords and reject invalid logins.	High
FR-1.3	JWT Token Management	The system shall issue a JWT on successful login and validate it on subsequent requests.	High
FR-1.4	Role-Based Authorization	The system shall restrict features and APIs according to the user's role.	High
FR-1.5	Protected API Access	The system shall deny access to protected APIs without a valid token.	High

MODULE 2 — Customer & Order Management

Requirement ID	Requirement	Description	Priority
FR-2.1	Customer Management	The system shall allow authorized users to create, view, update and delete customer records.	High
FR-2.2	Customer Order Creation	The system shall allow creation of a customer order with one or more order items.	High
FR-2.3	Order Validation	The system shall validate order data before the order proceeds in the workflow.	High
FR-2.4	Order Status Tracking	The system shall track and display the current status of each order through its lifecycle.	High

MODULE 3 — Product & BOM

Requirement ID	Requirement	Description	Priority
FR-3.1	Product Management	The system shall allow authorized users to create, view, update and delete products.	High
FR-3.2	BOM Creation	The system shall allow creation of a Bill of Materials (BOM) for a product.	High
FR-3.3	BOM Item Management	The system shall allow BOM items (materials and quantities) to be added, edited and removed.	High
FR-3.4	Material Requirement Calculation	The system shall compute required materials for an order from the order quantity and BOM quantities.	High

3. Functional Requirements (Continued)

MODULE 4 — Inventory & Procurement

Requirement ID	Requirement	Description	Priority
FR-4.1	Inventory Availability	The system shall display available stock for each material/product.	High
FR-4.2	Stock Movement	The system shall record stock movements that affect inventory.	High
FR-4.3	Shortage Detection	The system shall compare required and available quantities and identify shortages.	High
FR-4.4	Supplier Management	The system shall allow authorized users to manage supplier records.	Medium
FR-4.5	Purchase Order Creation	The system shall allow creation of purchase orders with items to cover shortages.	High
FR-4.6	Goods Receipt	The system shall record receipt of goods against a purchase order.	High
FR-4.7	Inventory Update	The system shall update inventory when goods are received.	High

MODULE 5 — Production

Requirement ID	Requirement	Description	Priority
FR-5.1	Production Order	The system shall allow creation of a production order for a customer order.	High
FR-5.2	Production Status	The system shall track production progress through status updates.	High
FR-5.3	Quality Check	The system shall record the quality check outcome for production output.	High
FR-5.4	Production Completion	The system shall mark production complete after the quality check.	High

MODULE 6 — Finance

Requirement ID	Requirement	Description	Priority
FR-6.1	Invoice Generation	The system shall generate an invoice for a completed order.	High
FR-6.2	Invoice Management	The system shall allow authorized users to view and manage invoices.	Medium
FR-6.3	Payment Recording	The system shall allow recording of payments against invoices.	High
FR-6.4	Payment Status	The system shall show the payment status of each invoice.	High

MODULE 7 — Reporting & Administration

Requirement ID	Requirement	Description	Priority
FR-7.1	Dashboard	The system shall present a summary view of order and operational status.	Medium
FR-7.2	Reports	The system shall provide reports on orders, inventory, procurement, production and finance data.	Medium
FR-7.3	Audit History	The system shall maintain and display an audit trail of significant user actions.	High
FR-7.4	Administrative Controls	The system shall allow the ADMIN to manage users and roles.	High

CORE BUSINESS LOGIC

Required Material = Order Quantity × BOM Quantity	Shortage = Required Material – Available Quantity (when Required exceeds Available)
--	--

Example: 100 Office Chairs	BOM Qty per Chair	Order Qty	Required Material
Seat	1	100	100
Frame	1	100	100
Wheels	5	100	500
Screws	10	100	1000

Example Shortage	Required	Available	Shortage
Frame	100	70	30

4. System Interfaces

4.1 Hardware Interfaces

Hardware	Use	Hardware	Use
Keyboard, Mouse	Data entry and navigation	Computer / Laptop	Client device running the browser
Display	Viewing the user interface	Network connection	Access to the application server

4.2 Software Interfaces

Interface	Technology	Purpose
Frontend	Angular	User interface
Backend	Spring Boot	Business logic / API
Database	MySQL	Persistent storage
ORM	JPA / Hibernate	Database mapping
Security	Spring Security / JWT	Authentication and authorization
Build	Maven	Backend build
API	REST / JSON	Communication between frontend and backend

4.3 Communication Interfaces

Aspect	Description
Protocol	REST over HTTP between Angular (HttpClient) and Spring Boot
HTTP Methods	GET (retrieve), POST (create), PUT (update), DELETE (remove)
Data Format	JSON request and response bodies
Authorization	JWT sent with each protected request in the Authorization header (Bearer scheme)

4.4 System Architecture

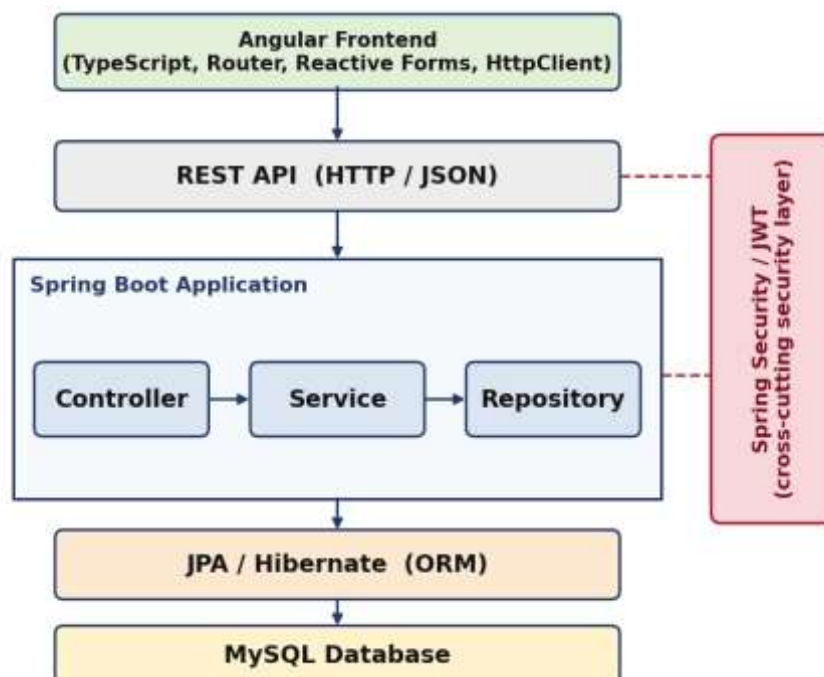


Figure 4.1 – Layered architecture with Spring Security / JWT as a cross-cutting security layer

Layer	Responsibility	Layer	Responsibility
Controller	Receives REST requests and returns JSON responses	Repository	Data access through Spring Data JPA
Service	Business logic and workflow rules	JPA / Hibernate	Maps Java entities to MySQL tables

5. Non-Functional Requirements (NFRs)

NFRs are stated qualitatively and are verified through inspection, review and testing of the delivered system.

NFR ID	Category	Requirement	Verification / Validation
NFR-1.1	5.1 Performance	Pages and API calls shall respond without noticeable delay under normal use.	Manual observation during functional test runs
NFR-1.2	5.1 Performance	Database access shall use JPA/Hibernate repositories rather than ad-hoc data access.	Code review of the repository layer
NFR-2.1	5.2 Security	Passwords shall be stored only as BCrypt hashes.	Inspect stored values; code review
NFR-2.2	5.2 Security	Protected APIs shall require a valid JWT.	Call APIs with and without a token
NFR-2.3	5.2 Security	Access shall be enforced by role (RBAC).	Test each role against restricted endpoints
NFR-2.4	5.2 Security	All inputs shall be validated (Bean Validation, Reactive Forms).	Submit invalid data; verify rejection
NFR-2.5	5.2 Security	Exceptions shall be handled and returned as controlled error responses.	Trigger error cases; inspect responses
NFR-2.6	5.2 Security	Significant user actions shall be recorded in the audit log.	Perform actions; inspect AUDIT_LOG
NFR-3.1	5.3 Reliability	Order, stock and financial data shall persist consistently in MySQL.	Verify data after create/update and restart
NFR-3.2	5.3 Reliability	Invalid operations shall fail with clear errors rather than corrupt data.	Negative test cases
NFR-4.1	5.4 Maintainability	Code shall follow Controller–Service–Repository layering, organized by module.	Code and package structure review
NFR-4.2	5.4 Maintainability	Modules shall be loosely coupled so features can change independently.	Code review
NFR-5.1	5.5 Portability	The backend and database shall run on any operating system supporting Java, Node.js and MySQL.	Run on the available platforms
NFR-5.2	5.5 Portability	The client shall run in modern web browsers without installation.	Open in supported browsers
NFR-6.1	5.6 Usability	The UI shall provide consistent navigation and role-appropriate menus.	Walkthrough per role
NFR-6.2	5.6 Usability	Forms shall display clear validation messages.	Inspect form behaviour

6. System Models and Database Design

6.1 Use Case Diagram



Figure 6.1 – Use cases by role

6.2 DFD Level 0



Figure 6.2 – Context-level data flow

6.3 ER Diagram

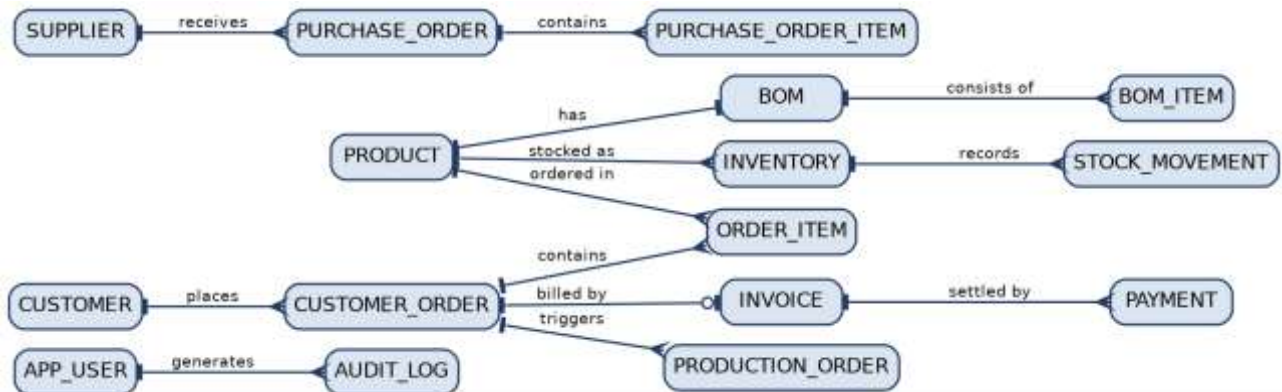


Figure 6.3 – Entity-relationship diagram (entities and relationships)

6.4 Activity Diagram



Figure 6.4 – Order processing activity flow

6.5 Sequence Diagram

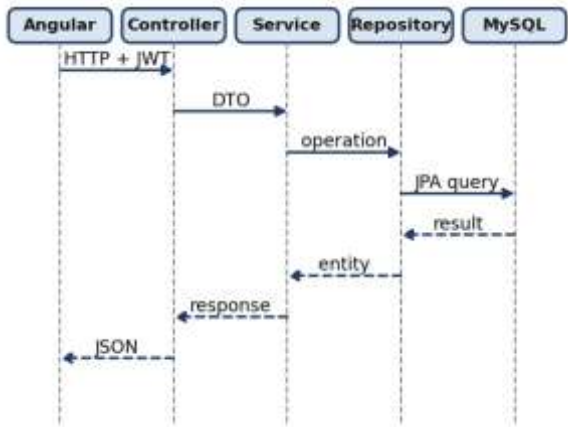


Figure 6.5 – Request flow across layers

6.6 Class Diagram

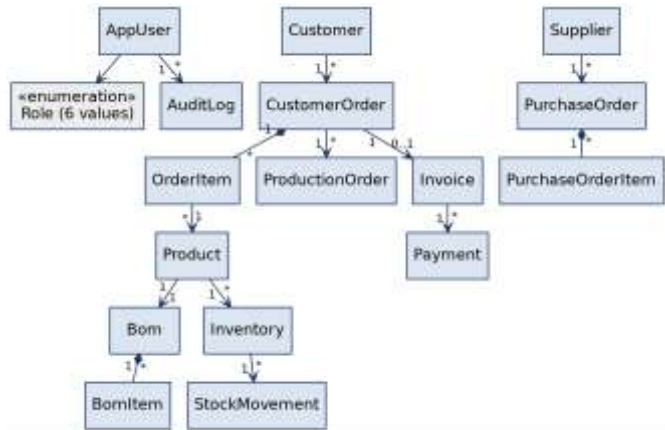


Figure 6.6 – Major domain classes and relationships

7. Testing Strategy

TC-ID	Module	Test Case	Expected Result	Status
TC-01	Login	Log in with valid credentials	JWT issued; user reaches dashboard	To Be Verified
TC-02	Invalid Login	Log in with wrong credentials	Access denied with error message	To Be Verified
TC-03	Authorization	Access an endpoint outside the user's role	Request rejected	To Be Verified
TC-04	Customer CRUD	Create, view, update, delete a customer	Operations succeed and persist	To Be Verified
TC-05	Product CRUD	Create, view, update, delete a product	Operations succeed and persist	To Be Verified
TC-06	BOM	Create a BOM and add BOM items	BOM saved with its items	To Be Verified
TC-07	Material Calculation	Calculate requirement for an order	Required = Order Qty × BOM Qty	To Be Verified
TC-08	Inventory	View stock availability	Correct quantities displayed	To Be Verified
TC-09	Shortage Detection	Place an order exceeding available stock	Shortage identified correctly	To Be Verified
TC-10	Purchase Order	Create a purchase order for a shortage	Purchase order saved with items	To Be Verified
TC-11	Goods Receipt	Record receipt against a purchase order	Receipt saved; inventory updated	To Be Verified
TC-12	Production	Create and update a production order	Status tracked correctly	To Be Verified
TC-13	Invoice	Generate invoice for a completed order	Invoice created	To Be Verified
TC-14	Payment	Record payment against an invoice	Payment saved; status updated	To Be Verified
TC-15	Validation	Submit invalid form/API data	Request rejected with message	To Be Verified
TC-16	Audit	Perform a tracked action	Audit entry recorded	To Be Verified

8. Security Summary

Authentication	JWT	Role Verification	Protected API	Validation	Audit
BCrypt-hashed credentials	Token issued at login	RBAC via Spring Security	No valid token → denied	Bean Validation, Reactive Forms	Actions logged in AUDIT_LOG

9. Conclusion

OrderCraft unifies the manufacturing order lifecycle in a single secure web application. It integrates customer and order management, BOM-driven material requirement calculation, inventory-based shortage detection, procurement, production tracking, quality checking and finance. A layered Spring Boot architecture, an Angular frontend and a MySQL database provide clear separation of concerns and maintainability, while JWT authentication, BCrypt hashing, role-based authorization, input validation and audit logging provide controlled and traceable access. The result is a structured, extensible foundation for end-to-end manufacturing order handling.

10. FUTURE SCOPE (not part of the current implementation)

AI-based demand forecasting	Barcode / QR integration	Mobile application	Cloud deployment
Advanced analytics	Supplier performance analytics	Automated notifications	Advanced production planning